Chapter 21 worked solutions

Image and Video Understanding

Exercises 1–4 work through image shapes, preprocessing, and attention counts by hand. Exercises 9–12 extend those ideas to video tokens and evaluation.

1. Make a patch grid

Read the image dimensions as height by width. The image is 12 pixels tall and 8 pixels wide, so a row of patches is 4 pixels high and holds two patches side by side. There are $12/4 = 3$ patch rows and $8/4 = 2$ patch columns, for a total of $3 \times 2 = 6$ patches.

Each patch contains $4 \times 4 = 16$ pixels, with three channel values per pixel. Flattening puts all $16 \times 3 = 48$ values into one row. For one image, the patch array has shape $[6, 48]$. The shared projection multiplies it by a weight matrix of shape $[48, 5]$, giving six patch tokens of width five. The classification token adds one more row of width five at the front. The resulting token array has shape $[7, 5]$.

2. Change the patch size

Imagine a small bright square in one corner of an 8×8 region. With patches of side eight, the bright square and its surroundings contribute to one token. With patches of side four, the same region produces four tokens. If the bright square falls within one of those smaller patches, attention can assign that patch a weight of its own.

For full attention over N patch tokens, each token can query all N keys, including its own. The score matrix has one row per query and one column per key, so it contains N^2 entries per head. These counts exclude the classification token.

Patch side	Patch grid	Patch tokens	Score entries per head
8	4×4	16	$16 \times 16 = 256$
4	8×8	64	$64 \times 64 = 4,096$

Halving the patch side doubles the number of positions along each axis. That makes four times as many tokens and sixteen times as many score entries.

3. Check an input convention

A channel value of 153 is 60 percent of the maximum possible value of 255. Divide by 255 first, because the mean and standard deviation in the exercise are expressed on the $[0, 1]$ scale. Subtracting the mean of 0.5 and dividing by the standard deviation of 0.25 gives

$$\frac{153/255 - 0.5}{0.25} = \frac{0.60 - 0.50}{0.25} = 0.40.$$

Skipping the scale conversion gives $(153 - 0.5)/0.25 = 610$. That calculation uses a raw channel value with statistics for scaled inputs.

Two other choices to match to the checkpoint are the channel order, such as RGB or BGR, and the resize and crop rule, such as resizing the shorter edge before taking a center crop of the expected size.

4. Count video connections

Follow the center patch in frame 2. Temporal attention groups it with the center patches from all four frames. Spatial attention groups it with all nine patches in frame 2. For the drawing, arrange each frame's patches in a 3×3 grid and number them from 1 to 9, left to right and top to bottom. Each pair (f, p) identifies frame f and patch p.

One temporal group at patch 5	One spatial group in frame 2
$(1, 5)$	$(2, 1)$ $(2, 2)$ $(2, 3)$
$(2, 5)$	$(2, 4)$ $(2, 5)$ $(2, 6)$
$(3, 5)$	$(2, 7)$ $(2, 8)$ $(2, 9)$
$(4, 5)$	

Each group is fully connected for its attention operation. Every member can query every member, including itself. Scores are for ordered query–key pairs. For example, the score with query $(1, 5)$ and key $(2, 5)$ is a separate entry from the score with those roles reversed.

Attention	Groups	Scores in each group per head	Total per head
Joint	1 clip	36×36	1,296
Temporal	9 patch locations	4×4	144
Spatial	4 frames	9×9	324

Joint attention puts all $4 \times 9 = 36$ patch tokens in one group. Divided attention performs the temporal and spatial operations in sequence. Adding their counts gives $144 + 324 = 468$ score entries, with one head counted in each operation. All counts exclude special tokens.

9. Form tubelets (optional)

Take the upper-left 4×4 patch from frame 1 and the patch at the same location in frame 2. Together they form one tubelet, a block that spans both space and time. Use the same patch grid for each frame and pair frames $(1, 2)$, $(3, 4)$, and $(5, 6)$ to form three nonoverlapping time groups.

Each 8×12 frame has $8/4 = 2$ patch rows and $12/4 = 3$ patch columns. Each frame pair contributes six tubelets, one for each patch location, so the clip produces $3 \times 2 \times 3 = 18$ tokens. A flattened tubelet contains $2 \times 4 \times 4 \times 3 = 96$ RGB values before projection.

Taking patches separately in every frame gives $6 \times 2 \times 3 = 36$ tokens, each with $4 \times 4 \times 3 = 48$ input values before projection. Tubelets halve the sequence length here by placing the pixels from two frames at the same location into one token.

10. Find a leaking split (optional)

Imagine that the first clip of recording R1 ends as a runner crosses a doorway. The next clip begins just before the crossing and shows it again. A random split by clip could put those two views in training and test. A classifier might recognize the same frames, the same runner, or the same doorway. Its test score would then answer a different question from how well it classifies a new recording.

For a new-recording evaluation, assign whole original recordings to splits before assigning their derived clips. For example, use R1–R7 for training, R8 for validation, and R9–R10 for test. With three clips per recording, that gives 21 training clips, 3 validation clips, and 6 test clips. All clips from R9 remain in test, even if they overlap or include an almost identical frame. Record the parent recording ID on each clip and check that the sets of parent IDs in the splits are disjoint. Use validation for tuning and open the test split after fixing the procedure. If recordings have labels, choose groups so the relevant classes are represented wherever possible. The group constraint takes priority over matching an exact percentage.

For a new-camera evaluation, make camera ID the grouping unit. Suppose camera A produced R1–R4, camera B produced R5–R7, camera C produced R8, and camera D produced R9–R10. Use cameras A and B for training, C for validation, and D for test. This example happens to give the same recording counts, but the grouping rule is stronger. A recording split could put a different recording from camera D into training, giving the model access to that camera's viewpoint, background, or image characteristics. Keep all recordings and clips from a camera together and verify disjoint camera IDs. With more cameras, hold out several and report performance by camera as well as overall. A single held-out camera tests transfer to that camera.

If the prediction target is a recording label, combine the three clip predictions by a rule fixed on validation, such as averaging their class scores and taking the highest. Report accuracy per recording. If the prediction target is a clip label, keep clip-level scoring and report the recording and camera counts too. The overlapping clips do not supply three independent measurements of transfer.

11. Test an action's direction (optional)

Start with a clip of a drawer opening. Reversing its frames gives a clip of the drawer closing. The two clips show the same sampled images, including the same open and closed endpoints. What changes is which endpoint comes first. Choose complete motions without cuts or a second action in the same clip, and check that the reversed sequence still has an unambiguous closing label. A noisy or incomplete reversal makes the target uncertain.

For each held-out recording, select a fixed list of frames, for example eight equally spaced frames spanning the movement. Keep the timestamps, and create the reversed input by reversing that exact list. Use the same crops and frame preprocessing for both models. An image-based baseline computes a class-score vector independently on each frame and averages those vectors before choosing a label. The temporal model sees the same frame features in order, with time positions attached, and uses attention across the ordered sequence. A shared frozen frame encoder and the same training and validation recordings make the comparison easier to interpret. Use the same clip-level labels and training loss for both heads. For the image-based head, apply the loss after averaging the frame scores. Keep each original and its reversal in the same recording split. Include native opening and closing examples so that a particular label does not always come from reversing a recorded motion.

Averaging has the same value when we reverse the summands. The image baseline therefore has the same score vector for an original clip and its reversal, up to numerical rounding. Use deterministic evaluation and a fixed tie rule. If it predicts *opening* for both, the original is correct and the reversal is wrong. A temporal model can learn to distinguish the direction because a visual state near the start and the same state near the end have different time positions. The test measures whether it uses that information.

Score each clip against its own label. Report accuracy separately on original and reversed inputs and report per-class results. Then report **pair accuracy**, the number of pairs with both labels correct divided by the number of pairs. The average-of-frames baseline gets exactly one clip correct per pair when it predicts one of the two labels and its prediction is invariant to order. Its pooled clip accuracy is 1/2 and its pair accuracy is zero. A flip alone earns no pair credit. Predicting *closing* for the original opening clip and *opening* for its reversal would flip the label and get both wrong. Report the total pairs and, when estimating uncertainty, keep the pair together when resampling recordings. Improvement on this test supports sensitivity to temporal order under the chosen sampling and training conditions.

12. Compare two visual checkpoints (optional)

Suppose checkpoint A uses 16×16 patches and checkpoint B uses 8×8 patches and was trained at higher resolution. Assume both accept 224×224 inputs. First compare A and B at that common resolution on exactly the same held-out images. Fix the decoded image, crop region, spatial resize, and evaluation score. For classification, one choice is the fraction of images whose highest scoring class matches the label, with the same class mapping. Use each checkpoint's required channel order, scaling and normalization. Those conventions keep its inputs compatible while both checkpoints see the same crop.

Report the paired difference on the same images. This comparison answers which checkpoint performs better when both receive the same view at the same resolution. The difference reflects the combined effects of patch size, training resolution, and other learned or architectural properties. The common resolution controls the evaluation input, and each model still creates its own patch grid.

Next choose a single checkpoint, for example A, and evaluate it at 224×224 and 320×320 if it supports both. Hold its weights and the held-out images fixed. Resize the same crop region to the two input sizes with the same resize algorithm. Use the checkpoint's supported method for position information on the new grid, such as interpolating a learned two-dimensional position grid, and record that choice. Score the same labels with the same metric. This comparison estimates the effect of changing the supported inference resolution for A, including the associated changes to its input and patch grid. Keeping training fixed limits this comparison to inference resolution.

For these illustrative patch sizes, the patch grid and token counts are as follows. The last two columns assume one classification token and full dense attention across all input tokens.

Setting	Patch grid	Patch tokens	Total tokens	Scores per head per layer
A, 224×224	14×14	196	197	38,809
B, 224×224	28×28	784	785	616,225
A, 320×320	20×20	400	401	160,801

The score counts square the total token count. In an actual checkpoint, count its real special tokens and use its actual attention pattern. Measure inference latency on the same hardware and software with the same batch size and precision. Warm up each setting, time repeated runs, and wait for accelerator work to finish before stopping the timer. Report the median and a high percentile, such as the 95th, along with the number of timed runs. State whether time includes decoding and resizing the image, or report preprocessing and model time separately. Report peak memory if it limits the feasible batch size. The token and score counts describe the computation's shape. The measured times tell how the actual implementation runs.